DAY 8 DSA

Complete Lecture-Based Study Notes

Based on the uploaded Day 8 lecture transcript • Trees, AVL balancing, threaded trees, heaps, heap sort, and merge sort

How to use these notes

Read the concept first, trace the small examples on paper, then type and test the code. The lecture emphasizes building algorithmic thinking—not merely memorizing a finished implementation. The code below is illustrative Java-style study code to make the ideas concrete; it is not claimed to be a verbatim copy of the instructor's code.

Lecture scope: The transcript reviews BST deletion and non-recursive postorder, then covers skewed/balanced trees, rotations and AVL properties, threaded-tree concepts, tree classifications and array representation, heaps and heap sort, and finally merge sort and its recursive implementation.

Contents

• 1. Quick review: BST deletion

• 2. Non-recursive postorder traversal

• 3. Skewed vs balanced binary search trees

• 4. Balance factor and AVL trees

• 5. Rotations: LL, RR, LR, RL

• 6. Threaded binary trees and tree classifications

• 7. Array representation of trees

• 8. Heap data structure

• 9. Heap sort

• 10. Merge sort

• 11. Complexity summary

• 12. Interview questions and practice set

1. Quick review: BST deletion

A binary search tree (BST) follows the ordering rule: values in the left subtree are smaller than the node; values in the right subtree are greater than or equal to it (the lecture's stated convention). When deleting a node, the key task is to reconnect the remaining subtree so it does not become disconnected/orphaned.

Three deletion cases

Case	What to reconnect	Reason
Node is a leaf	Set the parent's matching child reference to null; if the node is the root, set root = null.	No descendants to preserve.
Node has one child	Connect the parent directly to the node's only child. If deleting the root, root becomes that child.	The child's subtree must become reachable.
Node has two children	Find the inorder successor (leftmost node in the right subtree), copy its value into the target, then delete the successor from the right subtree.	The successor is the smallest value greater than the target, so it can safely replace the deleted key.

Important root check: The node being deleted may itself be the root. In that case there is no parent pointer to update; update the root reference directly.

Two-child deletion: successor walk

```
successorParent = target
successor = target.right

while successor.left != null:
    successorParent = successor
    successor = successor.left

target.data = successor.data

# The successor has no left child.
if successorParent.left == successor:
    successorParent.left = successor.right
else:
    successorParent.right = successor.right
```

This works because the inorder successor is the smallest value in the target's right subtree. It can have a right child, so reconnect that right child rather than simply dropping the successor.

2. Non-recursive postorder traversal

Postorder means Left → Right → Node. Unlike preorder (Node → Left → Right) and inorder (Left → Node → Right), postorder must wait until both subtrees are processed before printing the parent. This makes a one-stack iterative implementation trickier.

Why a stack is needed

• When walking down the left side, we must remember the nodes we have not printed yet.

• After the left subtree, we still need to visit the right subtree.

• Only after the right subtree has been processed can we print the node.

The lecture demonstrates a one-stack approach with a visited flag/state on each node. On the first time a node is popped, if its right child exists and has not been visited, push the node again and move to its right child. Otherwise print the node and mark it visited. This is a teaching approach; in production code, avoid permanently storing traversal state in tree nodes unless that design is intentional.

One-stack approach with a last-visited reference

A common alternative uses a reference to the last node completed. It avoids adding a visited field to every tree node.

```
void postorder(Node root) {
    Deque<Node> stack = new ArrayDeque<>();
    Node current = root;
    Node lastVisited = null;

    while (current != null || !stack.isEmpty()) {
        if (current != null) {
            stack.push(current);
            current = current.left;
        } else {
            Node peek = stack.peek();

            if (peek.right != null && lastVisited != peek.right) {
                current = peek.right;
            } else {
                System.out.print(peek.data + " ");
                lastVisited = stack.pop();
            }
        }
    }
}
```

Dry run for a small tree:

```
        2
       / \
      1   3

Postorder output: 1 3 2
```

Complexity: O(n) time because every node is processed a constant number of times. Auxiliary stack space is O(h), where h is tree height; in a skewed tree this can become O(n).

3. Skewed vs balanced binary search trees

A skewed BST grows mostly in one direction. In a left-skewed tree, nodes repeatedly have only left children; in a right-skewed tree, nodes repeatedly have only right children. Inserting already sorted values into an ordinary BST can create this shape.

```
Left-skewed:          More balanced:

      70                    40
     /                     /  \
    60                    20   60
   /                     / \   / \
  50                    10 30 50 70
 /
40
```

BST search, insertion and deletion are generally O(h), where h is height. A skewed tree has height proportional to n, so these operations can degrade to O(n). A balanced tree keeps height much smaller—roughly logarithmic for

well-balanced search trees—so operations can be O(log n).

Shape	Height trend	Search time
Skewed BST	O(n)	O(n) worst case
Balanced BST	O(log n)	O(log n) typical/worst-case for guaranteed balanced structures

4. Balance factor and AVL trees

Balance factor (BF) is defined in the lecture as:

```
BF(node) = height(left subtree) - height(right subtree)
```

• BF = 0: left and right subtree heights are equal.

• BF = +1: left subtree is one level taller.

• BF = -1: right subtree is one level taller.

For an AVL tree, every node must have a balance factor of -1, 0, or +1. If any node's balance factor falls outside this range, rotations are used to restore balance. The lecture uses height(null) = -1 and height(leaf) = 0; keep that convention consistent when calculating heights.

```
int height(Node node) {
    if (node == null) return -1;
    return 1 + Math.max(height(node.left), height(node.right));
}

int balanceFactor(Node node) {
    if (node == null) return 0;
    return height(node.left) - height(node.right);
}
```

AVL is a self-balancing BST. It keeps the BST ordering rule while using rotations after insertions/deletions to prevent the tree from becoming too tall.

5. Rotations: LL, RR, LR, RL

A rotation changes the local shape of a tree while preserving the inorder sorted order. The goal is to reduce height around an unbalanced node without breaking the BST property.

Four common imbalance patterns

Pattern	Where the new key was added	Fix
LL (left-left)	Left subtree of the left child	Right rotation at the unbalanced node
RR (right-right)	Right subtree of the right child	Left rotation at the unbalanced node
LR (left-right)	Right subtree of the left child	Left rotation at left child, then right rotation at unbalanced node
RL (right-left)	Left subtree of the right child	Right rotation at right child, then left rotation at unbalanced node

Right rotation

The former left child y becomes the local root. Its right subtree T3 becomes z's left subtree. The ordering remains valid because all values in T3 are greater than y and smaller than z.

Left rotation

```
     / \                      / \
    T1  y        --->        z  T4
       / \                  / \
      T2 T4                T1 T2
```

The former right child y becomes the local root. Its left subtree T2 becomes z's right subtree.

Java rotation helpers

```
Node rotateRight(Node z) {
    Node y = z.left;
    Node t3 = y.right;

    y.right = z;
    z.left = t3;

    // Update stored heights here if your Node stores height.
    return y;
}

Node rotateLeft(Node z) {
    Node y = z.right;
    Node t2 = y.left;

    y.left = z;
    z.right = t2;

    // Update stored heights here if your Node stores height.
    return y;
}
```

Always assign the returned node to the subtree root. If rotating the whole tree, update root; if rotating a child subtree, update the corresponding parent link.

Insertion sequence examples

• Insert 30, 20, 10: creates LL imbalance at 30 → right rotation.

• Insert 10, 20, 30: creates RR imbalance at 10 → left rotation.

• Insert 30, 10, 20: creates LR imbalance at 30 → left rotate 10, then right rotate 30.

• Insert 10, 30, 20: creates RL imbalance at 10 → right rotate 30, then left rotate 10.

For AVL insertion, search down as in a BST, insert the new leaf, update heights while returning up the recursion, compute balance factors, and apply the appropriate rotation case.

6. Threaded binary trees and tree classifications

The lecture introduces threaded binary trees after AVL/tree-classification discussion. In a regular binary tree, a null left/right pointer represents a missing child. In a threaded tree, one or both null pointers can instead be used to point to a node's inorder predecessor or successor, allowing traversal with less reliance on a stack or recursion.

• Single-threaded tree: threads replace null pointers in one direction (left or right).

• Double-threaded tree: threads may be used in both directions.

• Implementations need tags/flags to distinguish a real child pointer from a thread.

Do not treat a thread as a normal child link when recursively traversing the tree; first check its tag.

Common tree classifications

Term	Meaning
Full/proper binary tree	Every node has either 0 or 2 children.
Complete binary tree	All levels are full except possibly the last, which is filled from left to right.
Perfect binary tree	Every internal node has two children and all leaves are at the same depth.
Skewed binary tree	Nodes form a chain mainly to the left or right.

Term	Meaning
Balanced tree	Height is kept small according to the particular balance rule.

7. Array representation of trees

A complete binary tree can be stored compactly in an array. With zero-based indexing:

Relationship	Index formula
Node at index i	i
Left child	2*i + 1
Right child	2*i + 2
Parent (for i > 0)	(i - 1) / 2 using integer division

Example: for a node at index 2, its left child is at index 5 and its right child at index 6. Array representation is efficient for complete trees such as heaps, but wastes space for sparse or highly skewed trees because empty positions may be needed.

8. Heap data structure

A heap is a complete binary tree that satisfies a heap-order property. It is not a BST: the heap guarantees order only between each parent and its children, not a full left-versus-right search ordering.

Type	Property	Root contains
Max-heap	Parent value ≥ each child	Maximum value
Min-heap	Parent value ≤ each child	Minimum value

Max-heap example

```
Array: [90, 70, 80, 40, 50, 60]
Tree:
          90
        /    \
      70      80
     /  \    /
   40   50  60
```

Heap insertion (sift up)

• Insert the new value at the end of the array to preserve completeness.

• Compare it with its parent.

• For a max-heap, while child > parent, swap them and continue upward.

Remove maximum (sift down)

• Save the root maximum.

• Move the last array element to the root and reduce heap size.

• Compare the current node with its children; swap with the larger child if the heap property is violated.

• Continue downward until the property is restored.

For an array heap with zero-based indexing, parent = (i-1)/2, left = 2i+1, right = 2i+2. Insert and remove-root take O(log n); reading the root takes O(1).

9. Heap sort

Heap sort uses a max-heap to sort values in ascending order. First build a max-heap, then repeatedly move the maximum at the root to the end of the active array and restore the heap on the remaining prefix.

Algorithm outline

• Build a max-heap from the array.

• Swap the root with the last element in the current heap.

• Shrink the heap boundary by one; the final element is now in its correct sorted position.

• Sift down the new root to restore the max-heap.

• Repeat until the heap has size 1.

```
void heapSort(int[] a) {
    int n = a.length;

    // Build max-heap bottom-up.
    for (int i = n / 2 - 1; i >= 0; i--) {
        siftDown(a, n, i);
    }

    // Move maximum to the end repeatedly.
    for (int end = n - 1; end > 0; end--) {
        int temp = a[0];
        a[0] = a[end];
        a[end] = temp;

        siftDown(a, end, 0);
    }
}

void siftDown(int[] a, int size, int i) {
    while (true) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < size && a[left] > a[largest])
            largest = left;
        if (right < size && a[right] > a[largest])
            largest = right;

        if (largest == i) return;

        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        i = largest;
    }
}
```

Complexity: bottom-up heap construction is O(n); the repeated extraction phase is O(n log n), so total time is O(n log n) in best, average and worst cases. The in-place array version uses O(1) auxiliary space (ignoring call-stack space; the iterative siftDown above avoids recursion). Heap sort is not stable by default.

10. Merge sort

Merge sort is a divide-and-conquer sorting algorithm. It splits an array into smaller subarrays until each part contains one element, then merges sorted parts back together. The lecture's final portion focuses on recursively dividing, merging, and copying the sorted temporary array back into the original array.

Three steps

• Divide: split the array around the midpoint.

• Conquer: recursively sort the left and right halves.

• Combine: merge the two sorted halves into one sorted section.

Illustration

```
Input: [8, 3, 6, 2]

Split:  [8, 3]       [6, 2]
Split:  [8] [3]     [6] [2]
Merge:  [3, 8]       [2, 6]
Final:  [2, 3, 6, 8]
```

Java implementation

```
void mergeSort(int[] a, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSort(a, left, mid);
    mergeSort(a, mid + 1, right);
    merge(a, left, mid, right);
}

void merge(int[] a, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }

    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];

    for (int p = 0; p < temp.length; p++) {
        a[left + p] = temp[p];
    }
}
```

Call it as mergeSort(array, 0, array.length - 1). The base case left >= right means a subarray of zero or one element is already sorted. During merging, both halves are already sorted; repeatedly copy the smaller front element, then copy any remainder.

Merge sort complexity

Measure	Complexity	Why
Time (best/average/worst)	O(n log n)	There are log n levels of splitting, and O(n) total merge work per level.
Auxiliary space	O(n)	A temporary array is used for merging.
Stable?	Yes, if equal items are taken from the left half first	The <= comparison preserves the original order of equal keys.

11. Complexity summary

Operation / algorithm	Typical or guaranteed complexity	Worst case / note
BST search, insert, delete	O(h)	O(n) when skewed; O(log n) when balanced
AVL search, insert, delete	O(log n)	Tree height is kept logarithmic
Postorder traversal	O(n) time	O(h) stack space
Heap peek root	O(1)	Root is at index 0
Heap insertion	O(log n)	Sift up
Remove heap root	O(log n)	Sift down
Build heap bottom-up	O(n)	Not O(n log n)
Heap sort	O(n log n)	In-place; not stable by default

Operation / algorithm	Typical or guaranteed complexity	Worst case / note
Merge sort	O(n log n)	Uses O(n) auxiliary space; stable with correct merge

12. Interview questions and practice set

Quick interview answers

Q. Why can a BST become O(n)?
A. Because inserting ordered data can create a skewed tree whose height is proportional to the number of nodes.

Q. What is an AVL balance factor?
A. Height(left subtree) minus height(right subtree). For an AVL tree, each node's BF must be -1, 0, or +1.

Q. Why are rotations useful?
A. They restore balance while preserving the BST inorder ordering.

Q. How is a heap different from a BST?
A. A heap enforces parent-child priority; a BST enforces ordering across left and right subtrees.

Q. Why is a heap a good array candidate?
A. A complete binary tree maps naturally to consecutive array indices.

Q. Why does heap sort use a max-heap for ascending order?
A. The maximum is repeatedly placed at the array's last active position.

Q. What is merge sort's main idea?
A. Divide the array, recursively sort halves, then merge sorted halves.

Q. Why does merge sort need temporary space?
A. Merging two sorted ranges requires a safe place to combine values before copying them back.

Practice questions

• 1. Insert 30, 20, 10 into an AVL tree. Which imbalance occurs and what rotation fixes it?

• 2. Insert 10, 30, 20 into an AVL tree. Name the double rotation required.

• 3. Using zero-based heap indexing, find the parent, left child and right child indices for i = 5.

• 4. In a max-heap array [90, 70, 80, 40, 50, 60], what value is removed first?

• 5. State the time complexity of building a heap bottom-up.

• 6. What is the postorder traversal of a tree with root 2, left child 1 and right child 3?

• 7. Explain why search in a skewed BST can take O(n).

• 8. What are the base-case condition and recursive calls in merge sort?

• 9. How much extra space does the usual array merge sort need?

• 10. What is the difference between a threaded pointer and a normal child pointer?

Answer key

• 1. LL imbalance; right rotation at 30.

• 2. RL case; right rotation at 30, then left rotation at 10.

• 3. Parent index 2; left child 11; right child 12.

• 4. 90.

• 5. O(n).

• 6. 1, 3, 2.

• 7. The tree height can grow to n, so the search may inspect nodes along a chain.

• 8. Base case left >= right; recursively sort [left, mid] and [mid+1, right], then merge.

• 9. O(n) auxiliary space.

• 10. A normal child pointer links to a real child; a thread points to an inorder predecessor/successor and must be identified by a tag/flag.

Final revision sheet

Remember: BST operation cost = O(height). Skewed BST → O(n). AVL balance factor = left height – right height; allowed values –1, 0, +1. LL → right rotate; RR → left rotate; LR → left then right; RL → right then left. Heap is complete + heap-order; array children are 2i+1 and 2i+2. Heap sort = O(n log n), O(1) auxiliary space for the iterative in-place version. Merge sort = O(n log n) time and O(n) auxiliary space.

Scope note: The transcript covers the above concepts and worked explanations. Where exact instructor code was unclear in the transcript, the code here is a clean illustrative implementation for study, not a verbatim transcript.